ICLR 2024 | Giving Tabular Data Its Own Foundation Model

HKU, BAAI, and Creatify AI propose UniTabE, a universal pretraining protocol that doesn't care how your table is shaped

Pretraining reshaped natural language processing and computer vision on a simple bet: learn general knowledge once from a mountain of unlabeled data, then transfer it cheaply to many downstream tasks. Yet the format that data science lives in every day, the humble table of rows and columns, was largely left behind. Stock prediction, real-estate forecasting, credit scoring, insurance pricing, all of it runs on tables, and none of it has a widely adopted tabular foundation model.

The obstacle isn't a shortage of data; it's structure. Every table has a different schema: different column names, different data types, a different number of columns. A model trained on one table's structure usually can't be reused on another. That single fact is what has kept the pretrain-once-transfer-everywhere paradigm from taking hold in the tabular world.

UniTabE, published at ICLR 2024, sets out to close that gap. It proposes a schema-agnostic pretraining protocol: represent every cell of an arbitrary table in a uniform way with one small module, refine the whole table with a Transformer encoder, and adapt to new tasks through free-form text prompts and a shallow autoregressive decoder. Pretrained on roughly 13 billion samples scraped from Kaggle, the model ends up beating both the long-standing industry workhorse XGBoost and prior tabular models across a range of benchmarks.

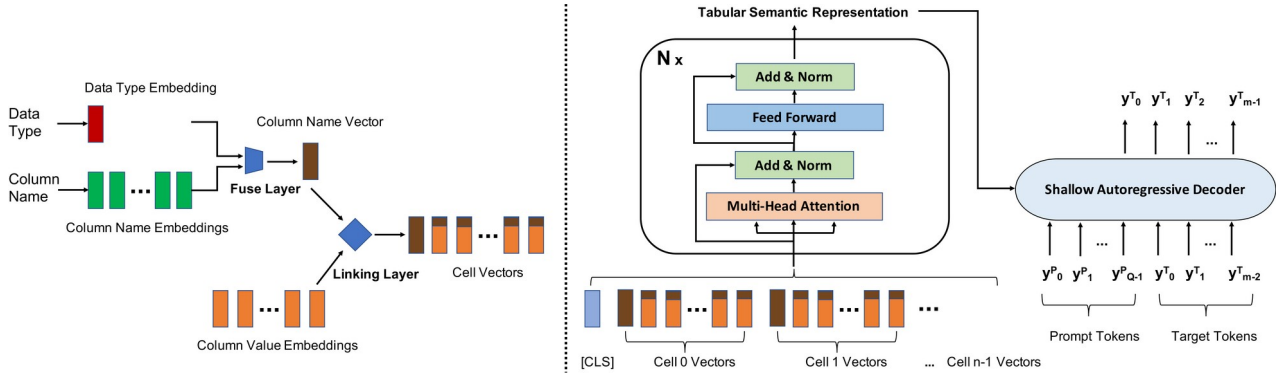

Figure 1. The UniTabE architecture. Left: the TabUnit module processing a single cell. A data-type embedding is gated into the column-name vector by the Fuse Layer, and the Linking Layer then injects that column vector into each value token to form the cell vector. Right: the full pipeline. All cell vectors are concatenated, prefixed with a [CLS] token, and passed through an N-layer Transformer encoder to produce a tabular semantic representation, which a shallow autoregressive decoder turns into the target token by token, guided by a free-form prompt.

Paper and code: the paper is on arXiv as 2307.09249 (https://arxiv.org/abs/2307.09249).

Why tables resist pretraining

Tables look simple, and that is exactly the problem: they are simple in too many different ways. Text and images arrive in a naturally uniform shape, a sentence is a sequence of tokens and an image is a grid of pixels, but a table's column names, column count, and per-column data types differ from one table to the next. Bake a specific structure into the architecture, and the next table forces a redesign.

Prior work mostly took one of two routes, each with a cost. One flattens the whole table into text and feeds it to a language model; that unifies the format but treats numbers as ordinary strings, discarding their quantitative meaning. The other requires the training and test tables to share the same fixed structure, which is brittle in the real world, where columns come and go over time. UniTabE aims for a third route: keep the cell-level structural information, but hard-code no fixed structure at all.

The core idea: treat every cell as a key-value pair

At the center of UniTabE is a small module called TabUnit that handles one cell at a time and reads it as a key-value pair: the column name is the key, the cell content is the value. The column name is embedded and mean-pooled into a column-name vector. In parallel, a separate data-type embedding, marking whether the column is numerical, categorical, or textual, is fused into that vector through a gating mechanism. This is the Fuse Layer, and its payoff is consistency: a 'salary' column is treated the same way whether its values are numbers or words like high, medium, and low.

The Linking Layer that follows injects the column-name vector into each of the column's value tokens, so the Transformer's self-attention knows which values belong to which column. All the cell vectors are concatenated, prefixed with a [CLS] token, and passed through an N-layer Transformer encoder to produce a semantic representation of the entire table. The decoder is deliberately shallow, an autoregressive LSTM or Transformer, that generates the answer token by token, conditioned on the [CLS] state and a free-form prompt such as 'fill in missing value, salary.' Keeping the decoder weak on purpose forces most of the learned knowledge into the reusable encoder.

Pretraining mixes two self-supervised signals. The first is multi-cell-masking: whole cells are randomly hidden and the model is trained to reconstruct their content by maximum log-likelihood. The second is contrastive learning: overlapping row blocks from the same example are pulled together while row blocks from different examples are pushed apart. Finetuning then reframes any downstream task as predicting a masked target column under a task-specific prompt, so pretraining and finetuning run through one and the same framework with no change to the architecture between the two stages.

What makes it possible: a 7TB tabular corpus

Pretraining at this scale needs a correspondingly large table collection. The authors assembled a roughly 7TB tabular corpus from Kaggle, spanning 303 domains, 283K tables, and about 13 billion examples. On average each table carries about 28.7 numerical columns, 0.4 categorical columns, and 7.7 textual columns; the largest domains are Investing (71K tables),

Time Series analysis (65K), Finance (52K), Economics (47K), and Games (32K). To avoid leakage, the Kaggle tasks used for evaluation were explicitly excluded from pretraining.

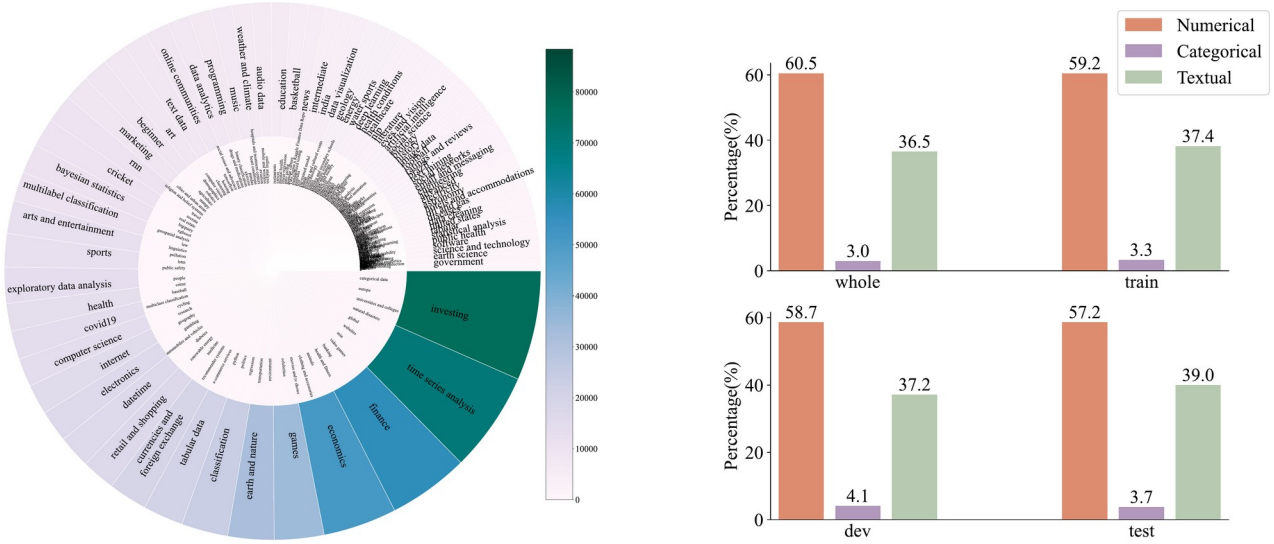

Figure 2. Distribution of the pretraining corpus. Left (a): a sunburst of table counts per domain, with Investing, Time Series, and Finance occupying the largest sectors. Right (b): the cell-level share of numerical, categorical, and textual data across the whole/train/dev/test splits. Numerical cells make up about 60%, textual about 37%, and categorical under 4%, so the corpus is dominated by numbers and text.

Does it work: clearing XGBoost on public benchmarks

Whether the pretraining pays off shows up in transfer. On seven widely used public tabular benchmarks, finetuned UniTabE reaches an average AUC of about 0.83, ahead of Tapas (0.81), FT-Transformer (0.80), and the industry's default choice, XGBoost (0.79). On 12 additional held-out Kaggle tasks (6 classification, 6 regression), UniTabE again outperforms XGBoost and the strong TransTab-LSTM baseline, on classification AUC and regression R2 alike. The reading here is straightforward: processing features at the cell level captures tabular structure better than simply flattening the table into a string of text.

Table 1. Average AUC on seven public tabular benchmarks (higher is better; scores averaged over five runs). Finetuned UniTabE leads every baseline at 0.83.

Method	Avg. AUC
XGBoost	0.79
NODE	0.74
AutoInt	0.79
Tapas	0.81
TaBERT	0.75
TabTransformer	0.79
FT-Transformer	0.80
TabNet	0.61
TUTA	0.76
TransTab	0.75
UniTabE (finetune)	0.83

Zero-shot, growing schemas, and missing values

The transfer that pretraining buys is clearest in a few scenarios closer to real data science. In the zero-shot setting, with no finetuning on the target data at all, the model still gives usable predictions on some datasets, for instance 0.94 AUC on IO and 0.89 on CA, well above random initialization. This suggests large-scale pretraining endows the model with genuinely transferable reasoning about tables rather than task-specific memorization.

A second, very practical scenario is tables that grow new columns. The authors simulate this by removing k columns from the training set and running inference on the untouched test set. Thanks to the flexibility of TabUnit, UniTabE adapts to added columns far more gracefully: removing a single column costs only a 3.5% AUC drop, less than half the 7.5% suffered by the structure-dependent TransTab-LSTM.

Table 2. Relative AUC drop under incremental columns (%, lower is better). Trained with k columns removed, then evaluated on the full test set; UniTabE loses the least.

Method	Drop 1 col	Drop 2 cols
TransTab-LSTM	7.5	11.7
UniTabE (scratch)	5.3	8.4
UniTabE (finetune)	3.5	5.8

Filling in a missing value slots into the same 'generate the target under a prompt' framing. UniTabE produces a regression prediction for numerical columns and generates text directly for textual ones, beating mean/mode baselines, linear regression, and even a GPT-3 comparison on both MAE and BLEU across several datasets, which shows the prompt-plus-generation framework covers heterogeneous imputation needs without special-casing.

Ablations: every component earns its place

The ablations confirm that nothing in the architecture is decorative. Removing the Linking Layer, which ties column names to their values, causes the single largest drop, from 0.83 to 0.75 average AUC; removing the data-type Fuse Layer also hurts, and removing both is worse still. Dropping either pretraining objective, multi-cell-masking or contrastive learning, lowers the score too, so the two signals are complementary. One finding stands out: a 1-layer decoder beats 3- and 6-layer decoders on downstream AUC, which supports the design choice of keeping the decoder shallow so the reusable knowledge stays in the encoder (deeper decoders score higher on generation BLEU but siphon that knowledge away).

Table 3. Ablation on the public benchmarks (average AUC, higher is better). Removing the Linking Layer hurts most, and both pretraining objectives contribute.

Configuration	Avg. AUC
UniTabE (full)	0.83
w/o Fuse Layer	0.77
w/o Linking Layer	0.75
w/o Fuse & Linking	0.72
w/o multi-cell-masking (MCM)	0.81
w/o contrastive learning (CL)	0.79

Takeaway: bringing the NLP pretraining recipe to structured tables

The message of UniTabE is that the pretraining paradigm which reshaped language and vision can be extended to structured tabular data, and the key is to respect the structure of tables rather than flattening them into text. Represent each cell faithfully with its column name, value, and data type; refine the whole table with a Transformer; adapt to new tasks through free-form prompts; pretrain on billions of examples. What comes out is a general tabular model that transfers across tasks, clears the XGBoost baseline, and gracefully handles messy realities such as missing values and tables that acquire new columns over time.

The result has honest boundaries. It rests on a particular Kaggle-sourced corpus, one dominated by numerical and textual columns with very few categorical ones, and the evaluation centers on classification, regression, imputation, zero-shot, and incremental columns; carrying the approach to the full breadth of tabular applications will take further validation. But as a solid step toward a genuine tabular foundation model, UniTabE gives a clear answer: when feature processing respects cell-level structure, pretraining works for tables too.